Claude Code Phone Bridge

Setup guide: follow, message, and approve Claude Code sessions on your computer from an Android phone

1. What it does

The bridge is a small program that runs on the computer where you use Claude Code. The phone app connects to it over your local network. With it you can:

- See all Claude Code sessions on the computer, and follow each one live.
- Send a message to a session.
- Allow or deny tool requests (for example, a shell command) from the phone.

The bridge uses only documented Claude Code interfaces: hooks, `claude agents`, and the Claude Agent SDK. Watching adds no tokens. Only a message that you send from the phone uses tokens, the same as a message typed on the computer. It works with any Claude Code sign-in, including Amazon Bedrock.

2. What you need

Item	Details
Computer	Windows, macOS, or Linux, with Claude Code installed and signed in. The `claude` command must work in a terminal.
Python	Python 3.10 or later.
Shell and curl	The hooks use `curl` and a POSIX shell. macOS and Linux have both. On Windows, Claude Code uses Git Bash, which includes them.
Claude Agent SDK	The Python package `claude-agent-sdk` (see step 3).
Phone	Android 8.0 or later, with the Bedrock Chat app (APK file).
Network	The phone and the computer must be on the same Wi-Fi or local network.

3. Set up the computer (one time)

3.1 Get the bridge files

Put these three files in one folder, for example `bridge`: `bridge.py`, `viewer.html`, and `install_hooks.py`. Open a terminal in the folder that contains `bridge`.

3.2 Install the Claude Agent SDK

```
python -m pip install claude-agent-sdk
```

If you do not want to install packages system-wide, install into a folder and tell the bridge where it is:

```
python -m pip install --target C:\Tools\claude-agent-sdk claude-agent-sdk
set CLAUDE_AGENT_SDK_DIR=C:\Tools\claude-agent-sdk          (Windows cmd)
export CLAUDE_AGENT_SDK_DIR=$HOME/tools/claude-agent-sdk     (macOS, Linux, Git Bash)
```

3.3 Add the hooks to Claude Code

The hooks tell the bridge what each session does. First run the script without options to see the change. It shows only the `hooks` section of your settings:

```
python -I bridge/install_hooks.py
```

Then apply it. The script makes a backup of `~/.claude/settings.json` first, and it keeps your other hooks:

```
python -I bridge/install_hooks.py --apply
```

> The hooks print nothing and do not slow Claude Code. When the bridge is not running, Claude Code works as usual. To remove the hooks, run `install_hooks.py --remove`.

4. Start the bridge

Start it each time you want to use the phone:

```
python -I bridge/bridge.py --lan
```

It prints the phone address and a pairing code:

```
Bridge running at http://localhost:8765
Phone address: http://192.168.1.20:8765
Pairing code: 703460 (valid 10 minutes)
```

- `--lan` lets the phone connect. Without it, only the computer itself can connect.
- A pairing code works one time and changes every 10 minutes. To see the current code, open `http://localhost:8765` in a browser on the computer and look at **Pair a phone**.
- On Windows, the first start can show a firewall prompt for Python. Allow it on **private** networks.

5. Set up the phone (one time)

1. Copy the APK file to the phone and open it. If Android asks, allow installs from that source.
2. Open the app, open the menu (three lines, top left), and tap **PC sessions**.
3. Enter the **phone address** from step 4 (for example `192.168.1.20:8765`) and the **pairing code**.
4. Tap **Pair**. The app keeps the pairing, so you do this one time only.

When to pair again

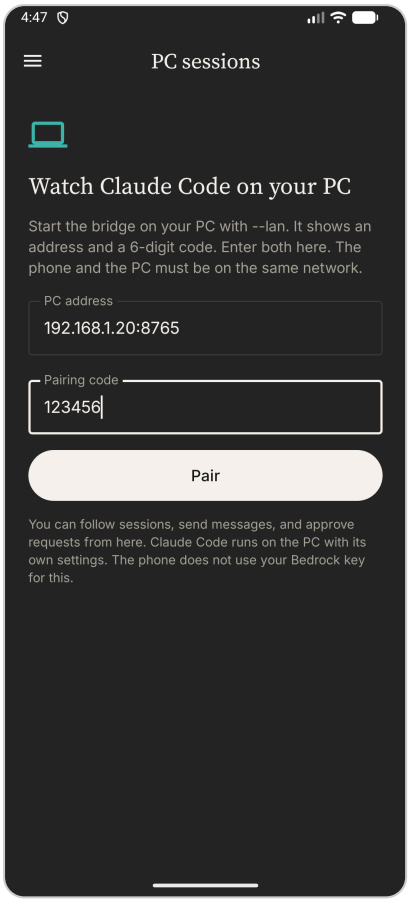
Restarting the bridge or the computer does **not** remove the pairing. The bridge keeps the phone's token in the file `bridge/.bridge-token` and reads it again at each start. You must pair again only in these cases:

Case	Why
You deleted `bridge/.bridge-token`	The bridge makes a new token, so the old one stops working.
You moved the bridge to a new folder or computer without that file	Same as above.
You tapped **Unpair this PC**, cleared the app's data, or reinstalled the app	The phone no longer has the token.
The computer's network address changed	The app keeps the old address. Unpair, then pair with the new address. To prevent this, reserve a fixed address for the computer in your router settings.

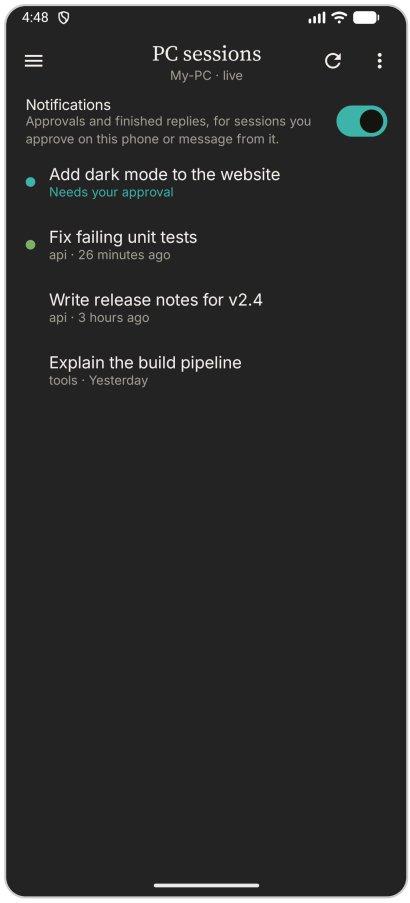
The bridge also keeps the messages that you sent from the phone (also the ones that still wait to send), and the messages that you typed on the computer while Claude worked, in `bridge/.session-prompts.json`. Claude Code's session list does not show these, so the phone needs this file to show them. If you move the bridge to a new folder or computer, copy this file with `.bridge-token`.

Messages and approvals also continue after a bridge restart, and each **Approve on this phone** switch stays as you set it. A phone message that waits for an open session also stays in the queue (it is kept in `.session-prompts.json`), and it goes in when Claude finishes its turn.

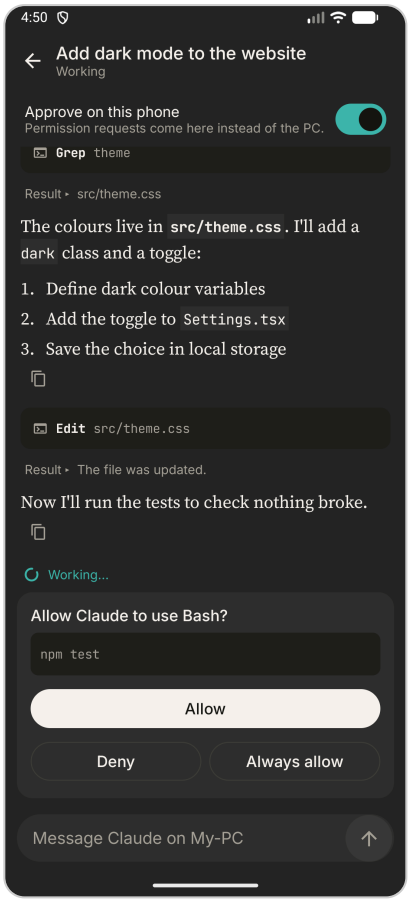
6. Use it



Pair the phone



All sessions



One session, with a request

Follow sessions

The list shows the sessions on the computer. A teal dot means the session is working or needs you. A green dot means it is open and waiting. Tap a session to follow it live. Tap a tool row or **Result** to see more.

Why a message to an open session is weaker

A message to a session that is open on the computer reaches Claude through a hook. Claude Code marks all hook text as "not user input", because any program could write it. So Claude takes such a message as a request, but it can ask you to confirm risky steps on the computer or through an approval on the phone. A message to a closed session is a normal prompt and does not have this limit. If you start the bridge from inside the Claude desktop app, the bridge removes the desktop app's own variables before it runs `claude -p`, so `claude` uses your `~/.claude/settings.json`.

Send a message

Type in the box at the bottom of a session and tap the arrow.

- **Session open on the computer:** the message goes in when Claude finishes its current turn. If you type on the computer first, your typed prompt goes first and the phone message waits.
- **Session closed:** the bridge continues the session on the computer with `claude -p --resume`.
- A session that was just opened or restarted on the computer listens for phone messages only after its first turn. Until then, a phone message waits and says so. Type anything on the computer once to

start it.
- If the session restarts just after a phone message went in, before Claude began work on it, the bridge puts the message back in the queue, so it is not lost.
- A message that waits shows "Sends when Claude finishes this turn". Tap **Delete** next to it to remove it before it goes in.

Session groups

If you group sessions in the sidebar of the Claude desktop app, the phone shows the same groups in the same order. Sessions in no group come last, under **Sessions**. Tap a group name to collapse or expand it. The bridge reads the groups from the desktop app's settings files and never changes them. Sessions started in a terminal are in no group.

The phone loads only your 40 most recent sessions. So a group shows only its sessions that are among those 40, and it can show fewer than the sidebar on the computer. Older sessions in a group are not listed on the phone.

The desktop app does not document these files, so an update can change them. If the bridge cannot find or read the groups, the phone shows the normal list without groups, and the bridge writes a short note in its window. Your sessions are not affected. To get the groups back after such an update, update `bridge.py`.

Context usage

The session header shows **Context** and a percentage: how much of the model's context window the session fills, from the tokens of Claude's last reply. Tap it to see the input, cache, and output tokens. Claude Code compacts the session by itself near the limit; use `/compact` or `/clear` on the computer to make room sooner. The window size comes from the model name, so a session with a smaller window can show a lower percentage than it is.

Stop a turn

While Claude works and the message box is empty, the send button changes to a stop button (a square). Tap it to stop the turn. Messages that wait to send are removed too.

- **Session open on the computer:** Claude Code cannot be interrupted from outside, so the stop takes effect at Claude's next tool call. The bridge refuses that call and tells Claude to end the turn. If Claude is only writing text, the turn ends when the text is done. A request that waits for approval on the phone is denied.
- **A run that the bridge started** (`claude -p --resume`): it ends at once.
- Stop needs the stop hook. If you set up the bridge before version 0.8.1, run `python -I bridge/install_hooks.py --apply` again.

Approve requests

Turn on **Approve on this phone** in a session. Then tool requests come to the phone as a card with **Deny**, **Always allow**, and **Allow**. When the switch is off, requests show on the computer as usual.

- If nobody answers on the phone in about 10 minutes, the request goes back to the computer.
- For a session that the bridge continues with `claude -p`, requests always come to the phone, and a request with no answer is denied.

- **Always allow** adds the rule that Claude Code suggests for that request.

Notifications

Notifications are on by default. Right after you pair, Android asks for two permissions: notifications, and background use ("Let app always run in the background?"). Allow both. Android does not let an app ask at install time. Then the phone notifies you even when the app is closed:

- **Approval needed:** a request waits for you on the phone. Tap the notification to open the session and answer.
- **Claude finished:** a turn ended, with the start of the reply.

To keep the phone quiet while you work at the computer, it notifies only for sessions that you control from the phone: sessions with **Approve on this phone** on, and turns that a phone message started. It does not notify about the session that is open on the screen.

- While notifications are on, a silent notification shows the connection: "Connected to *your PC*", or "Reconnecting…" when the phone cannot reach it. Android requires it for an app that stays connected in the background. Tap **Turn off alerts** on it, or use the **Notifications** switch at the top of the PC sessions list.
- Notifications need the bridge connection, so they work on the same network only.
- Some phones (for example OnePlus, Xiaomi, Samsung) also have their own battery limits. If notifications stop, set the app's battery use to **Unrestricted** in Android settings.
- Notifications use no tokens.

7. Token use

The bridge does not call Claude by itself. Tokens are used only when Claude does a turn. This table shows what each phone action costs, compared with the same action on the computer.

Phone action	Tokens
Open the app, see the session list, follow a session live	**None.** The bridge reads the session files on the computer. The hooks send events to the bridge only, and they add nothing to Claude's context.
Allow, Deny, or Always allow a request	**None extra.** It is the same as answering the dialog on the computer.
Notifications	**None.** They come from the same events as the live view.
Wait for a phone message (an idle open session)	**None.** A small background hook waits on the computer. It does not call Claude.
Send a message to an **open** session	**About the same as typing it on the computer.** The bridge adds a short note (about 25 tokens) that asks Claude to start its reply with your message as a quote. The quote is how the message shows on the computer. The reply is longer by that one line.
Send a message to a **closed** session	**Can be expensive.** The bridge continues the session with `claude -p --resume`. Claude reads the whole conversation again as input. For a long session, that can be many thousands of tokens. The app warns you before it sends.

> Claude Code caches the conversation for a short time (usually a few minutes). A message soon after the last turn reads most of the conversation from the cache, which costs less. After a long pause, the next turn reads it at the full price. This is the same for messages typed on the computer.

- Each phone message is one normal turn. The bridge does not retry a message or send it two times.
- An idle session that gets a phone message starts a new turn. Claude Code shows the message on the computer with the label "Stop hook blocking error". This label is normal and adds no tokens.
- To spend the fewest tokens, send messages to sessions that are open on the computer, and keep the messages short.

8. If something does not work

Problem	What to do
"Could not reach …"	Make sure the bridge runs with `--lan`, the phone is on the same network, and the address is correct. On Windows, check that the firewall allows Python on private networks.
"Wrong or expired code"	Get the current code from `http://localhost:8765` on the computer.
"This phone is no longer paired"	Pair again with a new code.
No sessions, or no live updates	Check that the hooks are applied (step 3.3) and that `claude agents --json` works in a terminal.
`ModuleNotFoundError: claude_agent_sdk`	Install the SDK (step 3.2), or set `CLAUDE_AGENT_SDK_DIR`.
Port 8765 is in use	Stop the other program. The hooks expect port 8765.

9. Security

- The phone gets a secret token when it pairs. Every request needs it.
- After 5 wrong codes, pairing is locked for 5 minutes.
- Hook events and pairing codes are accepted from the computer itself only.
- The connection is plain HTTP on your local network. Do not forward the port on your router or expose it to the internet.
- To unpair one phone, tap ⋮ → **Unpair this PC** in the app. To unpair all phones, stop the bridge, delete `bridge/.bridge-token` and `bridge/.paired-devices.json`, and start the bridge again.
- At start-up, the bridge lists the paired phones and when each was last seen. While it runs, it shows "Phone connected" and "Phone disconnected" when the app opens or leaves a PC screen. A phone that is not seen for 30 days leaves this list. This does not unpair it.

10. Remove it

```
python -I bridge/install_hooks.py --remove
```

Then delete the bridge folder and uninstall the app.